

Ex. 1

1. a) Recall the definition of the Jacobian and Hessian of a multi-variate function.

Jacobian Matrix: The Jacobian matrix of a vector-valued function is the matrix of all first-order partial derivatives.

Hessian Matrix: The Hessian matrix of a scalar-valued function is the matrix of all second-order partial derivatives.

b)

To minimize this quadratic approximation, we find the critical points by taking the gradient and setting it to zero:

$$\nabla f(x) \approx \nabla f(x_0) + \nabla^2 f(x_0)(x - x_0) = 0.$$

Solving for x , we get:

$$\nabla^2 f(x_0)(x - x_0) = -\nabla f(x_0).$$

$$x - x_0 = -(\nabla^2 f(x_0))^{-1} \nabla f(x_0).$$

Newton direction:

$$\Delta x = -(\nabla^2 f(x_0))^{-1} \nabla f(x_0).$$

2. a) Explain how the above strategy can be applied to gradient descent.

The above strategy can determine the step size and direction for each iteration.

Start with an initial guess x_0 . At each iteration k , compute the gradient $\nabla f(x_k)$ and the Hessian $\nabla^2 f(x_k)$ of the function at the current point x_k . Then calculate the Newton direction:

$$\Delta x_k = -(\nabla^2 f(x_k))^{-1} \nabla f(x_k).$$

After that, update the current point:

$$x_{k+1} = x_k + \alpha \Delta x_k,$$

where α is a step size parameter.

b) Considering the cost of inverting the Hessian, and the adjusted step-size in this version of gradient descent, discuss the speed up game provided by this approach.

Inverting the Hessian matrix is computationally expensive. The cost of inversion is $O(n^3)$. The Newton direction incorporates the curvature information of the function through the Hessian, leading to a more informed step size and direction. This can allow the algorithm to take larger and more accurate steps towards the minimum, potentially reducing the number of iterations needed to converge. The main advantage of using the Newton direction is the potential for quadratic

convergence. While the computation per iteration is more expensive due to the Hessian inversion, the total number of iterations required to achieve a given level of accuracy can be significantly reduced. For functions where the Hessian is well-conditioned and the cost of inversion is justified by the reduction in iterations, the Newton method can be much faster than gradient descent.

3. a) Determine the Jacobian of f in term of r .

First, the residual vector $r(\mathbf{w})$ is:

$$r(\mathbf{w}) = \begin{pmatrix} r_1(\mathbf{w}) \\ r_2(\mathbf{w}) \\ \vdots \\ r_m(\mathbf{w}) \end{pmatrix} = \begin{pmatrix} \mathbf{x}_1^\top \mathbf{w} - y_1 \\ \mathbf{x}_2^\top \mathbf{w} - y_2 \\ \vdots \\ \mathbf{x}_m^\top \mathbf{w} - y_m \end{pmatrix}.$$

The function $f(\mathbf{w})$ can be written as:

$$f(\mathbf{w}) = \frac{1}{2} r(\mathbf{w})^\top r(\mathbf{w}).$$

The Jacobian of f with respect to $\mathbf{w}$ is given by the gradient of f :

$$\nabla f(\mathbf{w}) = \nabla \left(\frac{1}{2} r(\mathbf{w})^\top r(\mathbf{w}) \right).$$

Using the chain rule, we get:

$$\nabla f(\mathbf{w}) = r(\mathbf{w})^\top \nabla r(\mathbf{w}).$$

The Jacobian matrix J of $r(\mathbf{w})$ with respect to $\mathbf{w}$ is:

$$J = \begin{pmatrix} \nabla r_1(\mathbf{w}) \\ \nabla r_2(\mathbf{w}) \\ \vdots \\ \nabla r_m(\mathbf{w}) \end{pmatrix} = \begin{pmatrix} \mathbf{x}_1^\top \\ \mathbf{x}_2^\top \\ \vdots \\ \mathbf{x}_m^\top \end{pmatrix}.$$

Thus, the gradient (Jacobian) of f in terms of r and J is:

$$\nabla f(\mathbf{w}) = J^\top r(\mathbf{w}).$$

b) Express the Hessian of f in term of its Jacobian.

The Hessian H of f is the matrix of second-order partial derivatives. It can be derived by differentiating the gradient:

$$H = \nabla^2 f(\mathbf{w}) = \nabla(\nabla f(\mathbf{w})).$$

From the previous part, we have:

$$\nabla f(\mathbf{w}) = J^\top r(\mathbf{w}).$$

$$H = \nabla(J^\top r(\mathbf{w})).$$

Since J is a constant matrix (independent of $\mathbf{w}$), we have:

$$H = J^\top \nabla r(\mathbf{w}).$$

$$H = J^\top J.$$

Thus, the Hessian of f in terms of its Jacobian is:

$$\nabla^2 f(\mathbf{w}) = J^\top J.$$

4. a) Show that $\mathbf{w} = -(\mathbf{J}^\top \mathbf{J})^{-1} \mathbf{J}^\top r(\mathbf{w})$

We know that:

$$\nabla f(\mathbf{w}) = \mathbf{J}^\top r(\mathbf{w}).$$

The Newton direction is given by:

$$\mathbf{w} = -(\nabla^2 f(\mathbf{w}))^{-1} \nabla f(\mathbf{w}).$$

The Hessian is:

$$\nabla^2 f(\mathbf{w}) = \mathbf{J}^\top \mathbf{J}.$$

Substituting these into the Newton direction formula, we get:

$$\mathbf{w} = -(\mathbf{J}^\top \mathbf{J})^{-1} \mathbf{J}^\top r(\mathbf{w}).$$

b) Applying SVD to $\mathbf{J}^\top \mathbf{J}$, show that there exist two orthogonal matrices $\mathbf{U}$ and $\mathbf{V}$, as well as a diagonal matrix $\mathbf{\Sigma}$, such that $\mathbf{w} = -\mathbf{V}(\mathbf{\Sigma}^\top \mathbf{\Sigma})^{-1} \mathbf{\Sigma}^\top \mathbf{U}^\top \mathbf{J}^\top r(\mathbf{w})$.

Applying Singular Value Decomposition (SVD) to $\mathbf{J}^\top \mathbf{J}$:

$$\mathbf{J}^\top \mathbf{J} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top,$$

where $\mathbf{U}$ and $\mathbf{V}$ are orthogonal matrices, and $\mathbf{\Sigma}$ is a diagonal matrix with singular values.

Thus,

$$(\mathbf{J}^\top \mathbf{J})^{-1} = (\mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top)^{-1} = \mathbf{V} \mathbf{\Sigma}^{-1} \mathbf{U}^\top.$$

$$\mathbf{w} = -(\mathbf{V} \mathbf{\Sigma}^{-1} \mathbf{U}^\top) \mathbf{J}^\top r(\mathbf{w}).$$

Since $\mathbf{J}^\top = \mathbf{U} \mathbf{\Sigma}^\top \mathbf{V}^\top$:

$$\mathbf{w} = -\mathbf{V}\mathbf{\Sigma}^{-1}\mathbf{U}^\top\mathbf{U}\mathbf{\Sigma}^\top\mathbf{V}^\top r(\mathbf{w}).$$

$$\mathbf{w} = -\mathbf{V}(\mathbf{\Sigma}^\top\mathbf{\Sigma})^{-1}\mathbf{\Sigma}^\top\mathbf{U}^\top\mathbf{J}^\top r(\mathbf{w}).$$

c) Show that if we appropriately truncate $\mathbf{U}$ into $\mathbf{U}_t$, then $\mathbf{w} = -\mathbf{V}\mathbf{\Sigma}_t^{-1}\mathbf{U}_t^\top\mathbf{J}^\top r(\mathbf{w})$.

If we truncate $\mathbf{U}$ to $\mathbf{U}_t$, this means we are only considering the top t singular values. Therefore, the corresponding truncation of $\mathbf{\Sigma}$ is $\mathbf{\Sigma}_t$.

$$\mathbf{J}^\top\mathbf{J} \approx \mathbf{U}_t\mathbf{\Sigma}_t\mathbf{V}^\top.$$

$$(\mathbf{J}^\top\mathbf{J})^{-1} \approx \mathbf{V}\mathbf{\Sigma}_t^{-1}\mathbf{U}_t^\top.$$

$$\mathbf{w} = -(\mathbf{V}\mathbf{\Sigma}_t^{-1}\mathbf{U}_t^\top)\mathbf{J}^\top r(\mathbf{w}).$$

Since $\mathbf{J}^\top = \mathbf{U}_t\mathbf{\Sigma}_t^\top\mathbf{V}^\top$:

$$\mathbf{w} = -\mathbf{V}\mathbf{\Sigma}_t^{-1}\mathbf{U}_t^\top\mathbf{U}_t\mathbf{\Sigma}_t^\top\mathbf{V}^\top r(\mathbf{w}).$$

This simplifies to:

$$\mathbf{w} = -\mathbf{V}\mathbf{\Sigma}_t^{-1}\mathbf{U}_t^\top\mathbf{J}^\top r(\mathbf{w}).$$

5. Based on the previous questions derive an algorithm to compute gradient descent.

- Choose an initial guess $\mathbf{w}_0$.
- Set tolerance ϵ and maximum number of iterations `max_iter`.
- For $k = 0, 1, 2, \dots, \text{max_iter}$:
 - a. Compute the Residual:

$$r(\mathbf{w}_k) = \begin{pmatrix} \mathbf{x}_1^\top \mathbf{w}_k - y_1 \\ \mathbf{x}_2^\top \mathbf{w}_k - y_2 \\ \vdots \\ \mathbf{x}_m^\top \mathbf{w}_k - y_m \end{pmatrix}.$$

- b. Compute the Jacobian:

$$\mathbf{J}_k = \begin{pmatrix} \mathbf{x}_1^\top \\ \mathbf{x}_2^\top \\ \vdots \\ \mathbf{x}_m^\top \end{pmatrix}.$$

- c. Compute Gradient:

$$\nabla f(\mathbf{w}_k) = \mathbf{J}_k^\top r(\mathbf{w}_k).$$

d. Compute Hessian Approximation:

$$\mathbf{H}_k = \mathbf{J}_k^\top \mathbf{J}_k.$$

e. SVD:

- Apply SVD to $\mathbf{H}_k$:

$$\mathbf{H}_k = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top.$$

- Truncate the matrices if necessary to $\mathbf{U}_t$ and $\mathbf{\Sigma}_t$.

f. Newton Direction:

$$\Delta \mathbf{w}_k = -\mathbf{V} \mathbf{\Sigma}_t^{-1} \mathbf{U}_t^\top \mathbf{J}_k^\top r(\mathbf{w}_k).$$

g. Line Search: Determine an appropriate step size α_k using line search techniques.

h. Update:

$$\mathbf{w}_{k+1} = \mathbf{w}_k + \alpha_k \Delta \mathbf{w}_k.$$

i. Convergence Check:

- If $\|\Delta \mathbf{w}_k\| < \epsilon$ or $\|\nabla f(\mathbf{w}_k)\| < \epsilon$, then stop the iteration.
- Return $\mathbf{w}_{\text{final}} = \mathbf{w}_k$.

```
def gradient_descent(initial_w, X, y, tol=1e-6, max_iter=500):
    w = initial_w
    for k in range(max_iter):
        r = X @ w - y
        J = X
        grad = J.T @ r
        H = J.T @ J
        U, Sigma, Vt = np.linalg.svd(H)
        Sigma_t = np.diag(Sigma)
        U_t = U
        delta_w = -Vt.T @ np.linalg.inv(Sigma_t) @ U_t.T @ J.T @ r
        alpha = line_search(f, grad, w, delta_w)
        w = w + alpha * delta_w
        if np.linalg.norm(delta_w) < tol or np.linalg.norm(grad) < tol:
            break
    return w
```

6. Without proving anything, explain why the above algorithm will perform better than gradient descent as introduced in the lectures.

The algorithm leverages Newton's method, which incorporates second-order information through the Hessian matrix, offering quadratic convergence and faster performance compared to the linear convergence of standard gradient descent. By using Singular Value Decomposition (SVD), the algorithm gains numerical stability and can handle ill-conditioned problems more effectively. Additionally, the adaptive step size determined by line search ensures optimal steps at each iteration, avoiding the pitfalls of fixed step sizes. This combination allows the algorithm to navigate complex optimization landscapes efficiently, reducing the total number of iterations and improving performance, especially in high-dimensional spaces.

Ex. 2

1. Sequential Algorithm

The c++ code of the algorithm is as follows:

```
int SequentialXOR(int A[], int n) {
    int S = 0;
    for (int i = 0; i < n; i++) {
        S = S ^ A[i];
    }
    return S;
}
```

The work of the algorithm is $O(n)$, because we traverse the entire array sequentially.

2. Parallel Algorithm

The parallel algorithm can be implemented using tree reduction. In each step, each processor performs the xor operation on two elements (from the previous step), resulting in half the elements.

```
void ParallelXOR(int A[], int n) {
    for (int d = 1; d <= log2(n); d++) {
        #pragma omp parallel for
        for (int i = 0; i < n; i += (1 << d)) {
            if (i + (1 << (d-1)) < n) {
                A[i] = A[i] ^ A[i + (1 << (d-1))];
            }
        }
    }
}
```

In each step of the parallel algorithm, we perform xor operations on $n/2^d$ elements, where d ranges from 1 to $\log(n)$. Thus, the total work is $O(n)$. The depth of the parallel algorithm is $O(\log(n))$, because it takes $\log(n)$ steps to complete all xor operations. The time complexity of the parallel algorithm is $O(\log(n))$ for the depth.

Using Brent's theorem, the speed of the algorithm can be expressed as $T_p \leq (W/P) + D$, where W is the work, P is the number of processors, and D is the depth. For this algorithm, we have $W = O(n)$ and $D = O(\log(n))$.

Assuming $P = n$ processors, $T_p \leq (n/n) + \log(n) = O(\log(n))$.

Therefore, the parallel algorithm can significantly reduce computation time from $O(n)$ to $O(\log(n))$ when there are enough processors available, indicating high efficiency in handling large datasets.

Ex. 3

Please refer to the corresponding code.

Speed Ranking (Fastest to Slowest):

- Stochastic Gradient Descent Using Hogwild!
- Stochastic Gradient Descent Without Hogwild!
- Batch Gradient Descent Using Broadcast Variables
- Batch Gradient Descent Without Broadcast Variables
- Steepest Gradient Descent With Broadcast Variables
- Steepest Gradient Descent Without Broadcast Variables

Stochastic Gradient Descent Using Hogwild! leverages parallel updates and doesn't wait for locks, making it the fastest.

Stochastic Gradient Descent Without Hogwild! updates weights after each sample, which is faster than batch methods due to its frequent updates, but not as fast as Hogwild!.

Batch Gradient Descent Using Broadcast Variables benefits from broadcasting the weights, reducing data shuffling and speeding up the process.

Batch Gradient Descent Without Broadcast Variables processes the entire dataset to update weights, making it slower than the broadcast version.

Steepest Gradient Descent methods require additional computation for the step size, which involves calculating the Hessian approximation, adding computational overhead. Using broadcast variables can mitigate some of this overhead, but not all.

Accuracy Ranking (Most Accurate to Least Accurate):

- Steepest Gradient Descent With Broadcast Variables
- Steepest Gradient Descent Without Broadcast Variables
- Batch Gradient Descent Using Broadcast Variables
- Batch Gradient Descent Without Broadcast Variables
- Stochastic Gradient Descent Without Hogwild!
- Stochastic Gradient Descent Using Hogwild!

Steepest Gradient Descent methods dynamically adjust the step size based on the gradient, leading to potentially more accurate convergence.

Batch Gradient Descent methods process the entire dataset for each update, leading to more stable and accurate updates compared to stochastic methods.

Stochastic Gradient Descent Without Hogwild! updates weights frequently, which can introduce more noise but can still converge to a good solution.

Stochastic Gradient Descent Using Hogwild! can introduce inconsistencies due to parallel updates without locking, potentially reducing accuracy.